

Universidad de Antioquia

Informática II

Informe final del proyecto

Skate en el Universo de los Dulces

Integrantes:

Jhonier Santiago Meñaca Santamaría

Didier Esneider Marín Gómez

Medellín, 2026

1. Introducción

Este documento presenta el desarrollo del videojuego Skate en el Universo de los Dulces, un proyecto realizado en C++ con Qt Creator. La idea principal fue construir un juego 2D con una temática llamativa, fácil de entender y con mecánicas suficientes para demostrar varios temas vistos en la materia, como programación orientada a objetos, manejo de eventos, animación, colisiones, uso de contenedores STL e inteligencia artificial sencilla.

La propuesta general nació como un videojuego ambientado en un mundo de dulces, donde el personaje principal recorre escenarios de chocolate, caramelo y gelatina mientras evita obstáculos y recoge premios. En el documento de diseño del proyecto se planteó que el Nivel 1 fuera un recorrido lateral y que el Nivel 2 fuera con una vista más centrada en supervivencia y un comportamiento diferente del entorno y del enemigo inteligente.

La motivación del proyecto fue crear algo visualmente atractivo, pero al mismo tiempo ordenado para ser programado en una estructura clara de clases. También se buscó que el juego tuviera una lógica fácil y aplicada por nosotros mismo

2. Contexto del proyecto, motivación y público objetivo

El proyecto se pensó como un videojuego sencillo pero completo, dirigido principalmente a estudiantes y a cualquier persona que quiera probar un juego 2D con mecánicas básicas de salto, esquivar y recolección. No se buscó hacer un juego demasiado complejo ni realista, sino uno que permitiera aplicar conceptos de programación de manera clara.

La temática de dulces se eligió porque facilita crear escenarios coloridos, obstáculos fácilmente reconocibles y elementos que se pueden distinguir sin dificultad. Además, esta temática nos daba mucha margen para inventar mecánicas simples, como superficies pegajosas o caramelos que caen desde arriba.

3. Planteamiento del problema

Durante el desarrollo de un videojuego, uno de los problemas principales es lograr que la lógica del juego sea clara, que los objetos se comporten de manera coherente y que el jugador entienda qué hacer sin necesidad de explicaciones largas.

En este proyecto se necesitaba un sistema que permitiera:

- controlar un personaje principal con movimiento y salto,
- generar obstáculos y premios con comportamientos distintos,
- detectar colisiones de forma correcta,
- mostrar información básica como vidas, puntos y tiempo,
- y cambiar la dinámica entre niveles para que el juego no se sintiera repetitivo.

La necesidad que cubre el software es la de tener una aplicación funcional y entretenida y que se use programación orientada a objetos y lógica de videojuegos en Qt. También organizar un proyecto relativamente grande en clases separadas y fáciles de mantener.

4. Definición general y objetivos

4.1. Definición general

Skate en el Universo de los Dulces es un videojuego 2D en el que un personaje tipo skater se mueve por escenarios hechos de dulces, enfrenta obstáculos y recoge elementos que suman puntos. El juego está dividido en dos niveles con dinámicas diferentes.

4.2. Objetivo general

Desarrollar un videojuego 2D en C++ y Qt que permita aplicar conceptos de programación orientada a objetos, manejo de eventos, físicas simples, contenedores STL e inteligencia artificial básica.

4.3. Objetivos específicos

- Crear una estructura de clases clara y modular.
- Implementar un personaje jugable con movimiento, salto, vidas y puntuación.
- Diseñar obstáculos y coleccionables con distintos comportamientos.
- Incorporar un sistema de colisiones funcional.
- Implementar una inteligencia artificial simple para generar obstáculos de forma dinámica.
- Diferenciar los niveles con mecánicas y vistas distintas.
- Presentar el juego con una interfaz entendible y llamativa.

5. Especificación de requerimientos

5.1. Requerimientos funcionales

- El usuario debe poder iniciar una partida desde el menú.
- El jugador debe moverse horizontalmente y saltar.
- El personaje debe tener sistema de vidas.
- El juego debe mostrar puntos y tiempo restante.

- Deben aparecer obstáculos y objetos coleccionables.
- El sistema debe detectar colisiones.
- El primer nivel debe usar una IA que genere obstáculos de forma variable.
- El segundo nivel debe cambiar la dinámica del juego.

5.2. Requerimientos no funcionales

- El juego debe ejecutarse de forma estable en Qt Creator.
- El código debe ser fácil de leer y mantener.
- La interfaz debe ser visualmente clara.
- El rendimiento debe ser suficiente para un juego 2D sencillo.

5.3. Restricciones

- El desarrollo debía ajustarse al tiempo disponible del semestre.
- No se buscó usar motores externos de videojuegos.
- Se trabajó únicamente con herramientas de C++ y Qt.

5.4. Usuarios destinatarios

El proyecto está pensado para usuarios jóvenes o estudiantes que quieran probar un juego sencillo y para profesores que evalúan el uso de conceptos de programación dentro de una aplicación funcional.

6. Metodología y planificación

Para el desarrollo se trabajó con una metodología sencilla, por etapas. Primero se definió la idea general del juego y se escribieron los requerimientos principales. Después se construyó la estructura de clases y, poco a poco, se fueron agregando las mecánicas de movimiento, colisión, inteligencia artificial y niveles.

Aunque el proyecto no siguió una metodología ágil formal, sí se trabajó de forma que cada parte se probó varias veces, se corrigieron errores y luego se agregaron mejoras. Esa forma de trabajo ayudó bastante porque en un videojuego muchas cosas solo se entienden bien cuando se prueban dentro del juego y no solo leyendo el código.

7. Diseño y arquitectura del sistema

La arquitectura del sistema se organizó de forma modular para separar responsabilidades. Esta decisión fue importante porque el juego tiene varias partes diferentes y cada una debía cumplir una función concreta.

7.1. Clase MainWindow

Esta clase controla la interfaz principal, los menús y la navegación entre pantallas y también se encarga de iniciar o reiniciar el juego según la opción seleccionada por el usuario.

7.2. Clase Jugador

La clase `Jugador` contiene toda la lógica del personaje principal: el movimiento, salto, gravedad, animaciones, vidas, puntos e invulnerabilidad temporal. Esta clase permite modificar el comportamiento del jugador sin afectar el resto del proyecto.

7.3. Clase Obstaculo

Esta clase representa los elementos que aparecen en el juego y que pueden ser dañinos o coleccionables y también maneja los modelos físicos utilizados para el movimiento de cada objeto.

7.4. Clase IAObstaculos

Esta clase administra el comportamiento inteligente del primer nivel. Decide cuándo generar objetos, qué tipo de objeto crear, qué física usar y qué tan seguido aparecen los elementos en pantalla.

7.5. Clase DronIA

Esta clase controla el comportamiento del enemigo del segundo nivel. Su propósito es perseguir o bloquear al jugador, haciendo que la dinámica sea distinta a la del primer nivel.

7.6. Relación entre clases

Las clases se comunican entre sí por medio de señales, slots y referencias directas controladas. El jugador actualiza sus estados; la IA escucha esos cambios y también se encarga de refrescar el HUD. Esta forma de trabajar evita mezclar demasiada lógica en una sola clase y hace más fácil entender el código.

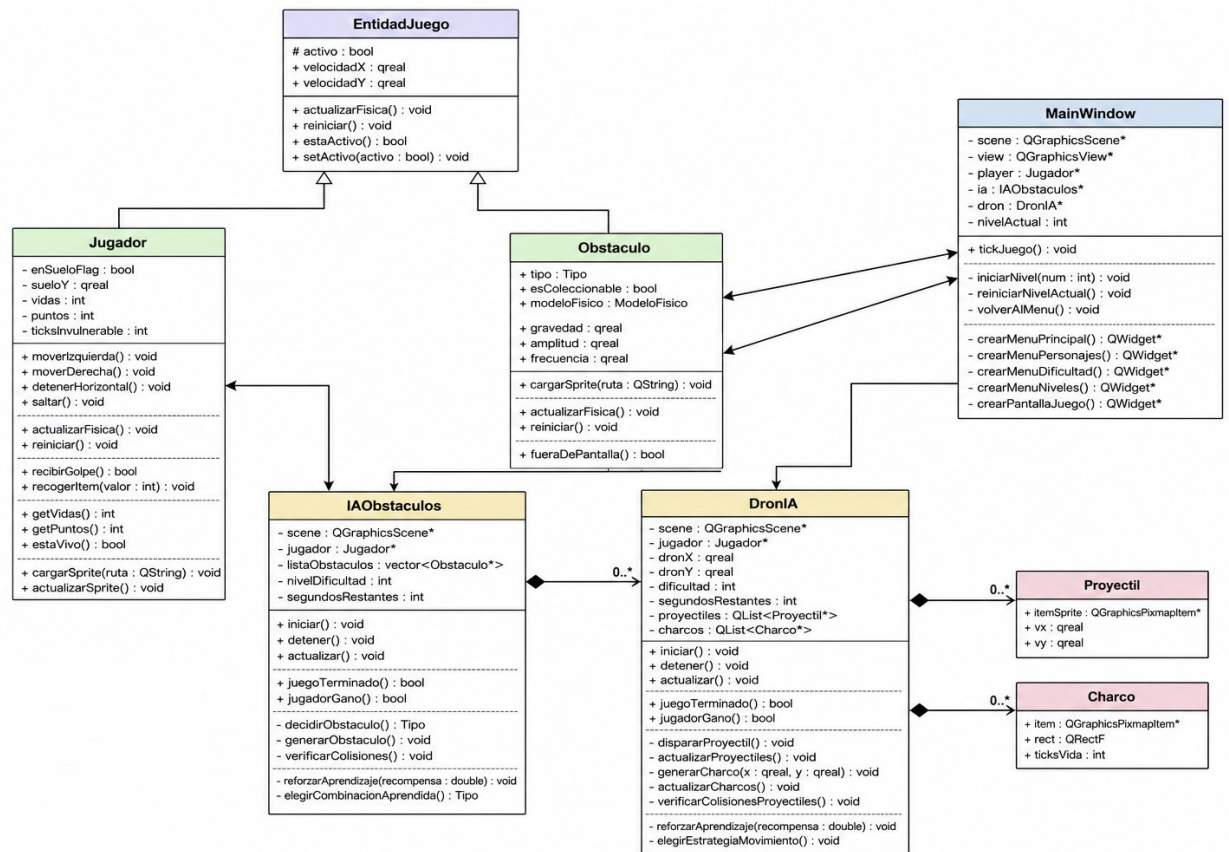


Figura 1: Diagrama de Clases

8. Dependencias externas y bibliotecas utilizadas

El proyecto se desarrolló usando las herramientas propias de Qt y la biblioteca estándar de C++. Las principales dependencias fueron:

- Qt Core, para temporizadores, tipos de dato y utilidades generales.
- Qt Widgets, para la interfaz gráfica.
- QGraphicsScene y clases relacionadas, para manejar la escena del juego.
- STL de C++, especialmente `vector` y `map`.

También se usaron recursos externos como imágenes de sprites y fondos para darle identidad visual al juego.

9. Desarrollo e implementación

9.1. Tipos de variables

Una de las primeras decisiones importantes fue elegir correctamente los tipos de variables. Se usó `qreal` para posiciones y velocidades porque el movimiento del juego necesita

decimales y no solo números enteros así que es útil para que el desplazamiento sea más suave.

Se usó `int` para vidas, puntos y tiempo porque esos datos son contables. También se usó `bool` para estados como estar vivo, estar en el suelo o saber si un objeto es coleccionable.

Para representar categorías distintas se usaron `enum`, porque así el código queda más ordenado y más fácil de leer. En lugar de poner números sueltos, se usan nombres como `FIS_LINEAL` o `FIS_PARABOLICA`, lo cual ayuda bastante a entender el programa y nos ayudo bastante.

9.2. Contenedores STL

El proyecto usa vector para almacenar los obstáculos activos en pantalla tuvimos esta decisión porque los obstáculos se generan y eliminan constantemente, así que necesitábamos una estructura dinámica y fácil de recorrer.

También se usa map para guardar los pesos de aprendizaje de la IA. Allí se asocia una combinación de tipo de obstáculo y modelo físico con un valor numérico. Esa solución permitió llevar un pequeño registro de qué combinaciones son más efectivas contra el jugador.

9.3. Sistema de señales y slots

Qt facilita mucho la comunicación entre clases gracias a señales y slots. En este proyecto se usó ese mecanismo para actualizar el HUD cuando cambian las vidas o los puntos del jugador decidimos elegir este método porque reduce el acoplamiento entre clases y evita que una clase dependa demasiado de otra.

9.4. Lógica del primer nivel

El primer nivel funciona con una IA sencilla. Primero observa el estado del jugador, después decide qué hacer, luego crea el obstáculo y finalmente aprende si esa combinación fue efectiva.

Esta secuencia se diseñó para que el juego tuviera una dificultad variable y para que no todos los obstáculos aparecieran siempre de la misma forma. Así el jugador no aprende una sola rutina fija, sino que debe adaptarse a lo que va pasando.

9.5. Temporizadores

Los temporizadores fueron fundamentales para controlar el ritmo del juego. Uno se encarga de generar obstáculos de forma periódica y otro controla el tiempo restante de la partida. Esta decisión fue importante porque en un juego en tiempo real no basta con ejecutar una sola vez la lógica; todo debe ir actualizándose constantemente.

9.6. Segundo nivel

Para el segundo nivel se decidió cambiar completamente la dinámica. En vez de repetir lo mismo se implementó un enemigo autónomo llamado DronIA. La idea fue que el segundo nivel se sintiera distinto y mostrara otra forma de jugar dentro del mismo proyecto.

En la propuesta original del Momento II se planteó un segundo nivel con vista cenital, supervivencia por tiempo y un entorno de gelatina más caótico. Con el avance del desarrollo varias ideas se simplificaron o se ajustaron por tiempo, pero se mantuvo la intención de que el segundo nivel fuera claramente diferente al primero.

9.7. Colisiones

Durante el desarrollo se tuvieron que revisar varias veces las colisiones. En algunos casos, el impacto visual no coincidía exactamente con la detección real, por lo que fue necesario ajustar posiciones y tamaños de los objetos. Después de varias pruebas se logró un comportamiento más estable y más justo para el jugador.

9.8. Balance de dificultad

También fue importante ajustar el balance del juego. Si los objetos aparecían muy rápido, el juego se volvía injusto. Si aparecían demasiado lento, el juego se volvía aburrido. Por eso se fueron probando diferentes velocidades, intervalos y frecuencias de aparición hasta conseguir una experiencia más equilibrada.

10. Procedimientos de prueba

Las pruebas del juego se realizaron de manera manual, ejecutando varias partidas y revisando el comportamiento de cada nivel. Algunos casos de prueba fueron:

- Verificar que el jugador pudiera moverse correctamente.
- Comprobar que el salto funcionara bien.
- Revisar que las vidas disminuyeran al colisionar con un obstáculo.
- Confirmar que los coleccionables sumaran puntos.
- Probar que el HUD mostrara el tiempo, las vidas y los puntos.
- Revisar que la IA generara obstáculos con variedad.
- Verificar que el segundo nivel se sintiera diferente al primero.

Los resultados obtenidos fueron positivos en general, aunque durante el desarrollo aparecieron errores pequeños que se fueron corrigiendo con pruebas repetidas.

11. Resultados y discusión

Al final se logró construir un videojuego funcional con dos niveles, un personaje jugable, un sistema de vidas, puntuación, obstáculos inteligentes y un enemigo autónomo. El proyecto cumplió con la idea general de mezclar programación orientada a objetos con una experiencia de juego sencilla pero entretenida.

Una de las fortalezas del trabajo fue la organización del código. Al separar el sistema en clases fue posible entender mejor cada parte y corregir errores con más facilidad. Otra fortaleza fue la temática del juego porque permitió hacer un proyecto visualmente agradable y diferente.

Como limitaciones, se puede mencionar que algunas ideas iniciales tuvieron que simplificarse por tiempo y dificultad. También hubo ajustes importantes en colisiones y balance del juego, porque al principio algunos obstáculos se sentían injustos o difíciles de prever.

Aun así, el resultado final fue satisfactorio porque el proyecto quedó estable, jugable y coherente con lo que se buscaba desde el inicio.

12. Conclusiones

Este proyecto permitió aplicar muchos conceptos de programación en una sola aplicación. Aprendimos a organizar mejor el código, a separar responsabilidades entre clases, a usar contenedores como vector y map, y a manejar eventos y señales de Qt de forma más ordenada.

También fue una experiencia útil para entender que en un videojuego no solo importa que el código compile, sino que el juego se sienta bien al jugarse. Por eso se hicieron varios ajustes de colisiones, dificultad y comportamiento de los enemigos.

En general, consideramos que el proyecto cumplió con los objetivos planteados y dejó una base sólida para seguir mejorándolo en el futuro.